HospSim

Un lenguaje para simular la atención hospitalaria

Ciencias de la Computación · Sección 4889

Profesor: JOSE LUIS SONCCO ALVAREZ

Teoría de Compiladores (1ACC0218) · 2026-2 · UPC

Trabajo Final – Hito 2: Segundo avance

Alvaro Jerry Rivera Calderón (u20231h075)

Ericks Santiago Alarcon Castellanos (U202322704)

Rodrigo Aaron Colonio Soto (u202323598)

Repositorio: https://github.com/Jere348/HospSim

Problemática y motivación

- La espera de un paciente depende de decisiones de capacidad: médicos por turno, camas, frecuencia de llegadas.
- Hoy se decide por experiencia u hojas de cálculo que no modelan colas.
- La simulación de eventos discretos responde estas preguntas, pero exige licencias o saber programar.
- HospSim: un lenguaje en español, con vocabulario hospitalario, que valida el modelo y entrega métricas.

```
recurso medicos = 1;
proceso atencion {
  etapa consulta usa medicos
        durante 15 minutos;
}
generar pacientes cada 5 minutos
        hacia atencion;
simular durante 1 hora;
mostrar pacientes_atendidos; // 4
mostrar espera_promedio;   // 15.00
mostrar ocupacion(medicos);// 100 %
```

Objetivos

Diseñar e implementar un compilador que traduzca descripciones de procesos hospitalarios en una simulación ejecutable con métricas de desempeño.

- Léxico y sintaxis en ANTLR4 con una gramática no ambigua (E, T, F).
- Análisis semántico con tabla de símbolos por ámbitos: 21 verificaciones (mínimo exigido: 6).
- Representación intermedia independiente de ANTLR y back end que ejecuta la simulación.
- Emisión de código intermedio textual con --ir y --guardar-ir.
- Validación automática con salidas esperadas calculadas a mano.

Construcciones del lenguaje

Construcción	Ejemplo
Declaración de recursos	`recurso camas = 20;`
Declaración de procesos	`proceso consulta_externa { etapa consulta usa medicos durante 20 minutos; }`
Declaración de etapas	`etapa consulta usa medicos durante 15 minutos;`
Generación de llegadas	`generar pacientes cada 12 minutos hacia consulta_externa;`
Simulación	`simular durante 12 horas;`
Consulta de métricas	`mostrar espera_promedio;`
Expresiones aritméticas	`2 * 10`

Analizador léxico

- 15 palabras reservadas (recurso, proceso, etapa, usa, durante, generar, pacientes, cada, hacia, simular, mostrar, minuto(s), hora(s)).
- ID: [a-zA-Z_][a-zA-Z0-9_]*
- NUMBER: [0-9]+ ('.' [0-9]+)?
- Operadores + − * /, signos = ; { } ( ).
- Espacios y comentarios // se descartan.
- Palabras reservadas antes que ID; coincidencia más larga.

```
$ python3 main.py --tokens prog.txt
1:0    RECURSO    'recurso'
1:8    ID         'medicos'
1:16   ASSIGN     '='
1:18   NUMBER     '1'
1:19   SEMI       ';'
3:0    PROCESO    'proceso'
3:8    ID         'atencion'
3:17   LBRACE     '{'
4:4    ETAPA      'etapa'
4:10   ID         'consulta'
4:19   USA        'usa'
4:23   ID         'medicos'
...
```

Gramática (notación del curso)

```
programa     → sentencias
sentencias   → sentencia sentencias  |  ε
sentencia    → decl_recurso  |  decl_proceso  |  sent_llegada  |  sent_simular  |  sent_mostrar
decl_recurso → recurso id = E ;
decl_proceso → proceso id { etapas }
etapas       → decl_etapa etapas  |  ε
decl_etapa   → etapa id usa id durante E unidad ;
sent_llegada → generar pacientes cada E unidad hacia id ;
sent_simular → simular durante E unidad ;
sent_mostrar → mostrar id ;  |  mostrar id ( id ) ;
unidad       → minuto  |  minutos  |  hora  |  horas
E            → E + T  |  E - T  |  T
T            → T * F  |  T / F  |  F
F            → ( E )  |  num  |  id
```

Gramática actualizada: precedencia y asociatividad

- Hito 1: expr → expr (+|−) expr | expr (*|/) expr — ambigua.
- ANTLR da prioridad a la primera alternativa: 2 + 3 * 4 = (2 + 3) * 4 = 20.
- Hito 2: gramática estratificada E → T → F, no ambigua.
- 2 + 3 * 4 = 14 y 20 − 5 − 3 = 12 (asociatividad izquierda).
- Cubierto por sim_precedencia y sim_asociatividad.

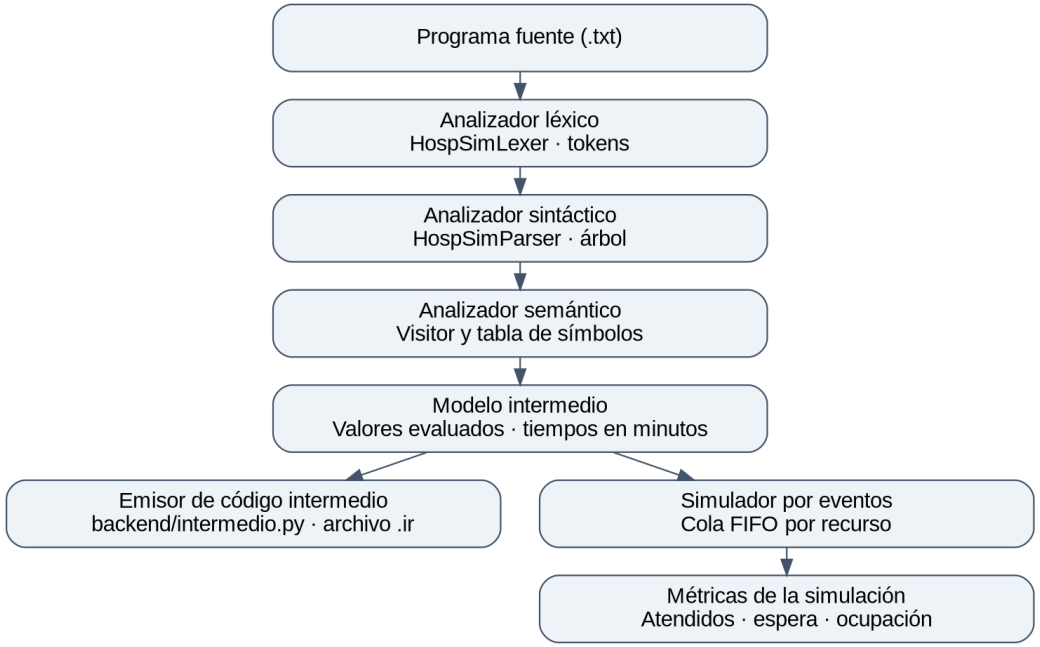
```
E
⇒lm E + T
⇒lm T + T
⇒lm F + T
⇒lm num + T
⇒lm num + T * F
⇒lm num + F * F
⇒lm num + num * F
⇒lm num + num * num
```

Analizador semántico: 21 verificaciones

Tabla de símbolos con ámbito global y uno por proceso · métricas predeclaradas · expresiones evaluadas en compilación

1. Identificador duplicado en el mismo ámbito
2. Recurso no declarado en una etapa
3. Lo usado en una etapa no es un recurso
4. Proceso destino no declarado
5. El destino de una llegada no es un proceso
6. Identificador no declarado en una expresión
7. Identificador que no es recurso en una expresión
8. División entre cero
9. Capacidad menor o igual a 0
10. Capacidad no entera
11. Duración de etapa menor o igual a 0
12. Intervalo de llegada menor o igual a 0
13. Duración de simulación menor o igual a 0
14. Proceso sin etapas
15. Falta la sentencia simular
16. Más de una sentencia simular
17. mostrar sobre algo que no es métrica
18. ocupacion sin recurso
19. Métrica que no recibe argumentos
20. ocupacion sobre algo que no es recurso
21. ocupacion sobre un recurso inexistente

Arquitectura del compilador



- Fases encadenadas en main.py; ante errores se detiene y reporta línea y columna.
- El back end solo recibe árboles válidos.
- Modelo intermedio con emisión textual y ejecución por eventos.
- Simulación por eventos con una cola FIFO por recurso compartido.

Plan de validación y resultados

- Golden files: tests/<caso>.txt + tests/esperado/<caso>.out.
- python run_tests.py comprueba salidas y códigos de retorno.
- Cubre fases léxica, sintáctica, semántica, gramática y back end.
- Métricas del back end calculadas a mano antes de aceptarlas.
- 20 casos originales y 11 regresiones: todos OK.

```
Caso: sim_dos_etapas
triaje 5 min (1 enfermero)
consulta 12 min (1 médico)
llegada cada 10 min, 60 min

consultas 5–17, 17–29, 29–41, 41–53
esperas 0, 2, 4, 6

pacientes_atendidos = 4
espera_promedio    = 3.00
ocupacion(enfermeros) = 50.00 %
ocupacion(medicos)    = 91.67 %
```

Estado del avance y próximos pasos

Componente	Estado
Analizador léxico y sintáctico	Completo
Analizador semántico (21 verificaciones)	Completo
Representación intermedia	Completa
Back end de simulación	Completo
Pruebas automáticas	Completas
Código intermedio textual	Implementado

- Código intermedio textual implementado. No se requiere LLVM.
- El emisor y el simulador utilizan el mismo Modelo validado.
- Próximo paso: ampliar escenarios y validar rendimiento.

Demostración

```
$ python3 main.py tests/entrada_completa.txt
Análisis semántico: OK
=== RESULTADOS ===
  pacientes_atendidos = 35
  ocupacion(medicos) = 99.48 %
  ocupacion(camas) = 95.83 %
  espera_promedio = 105.00 minutos

$ python3 main.py tests/error_tipos.txt
  [Error Semántico, línea 3] 'm' no es un proceso válido de destino
  ...

$ python run_tests.py
  20 casos + 11 regresiones: OK
```